**Rapport de Travaux Pratiques**

# Simulation Multi-Agent avec Mesa 3

**Formation de l'opinion publique**

**Étudiant :** *Ayman Joundi*
**Filière :** Master AIDC
**Encadrant :** *Lhoussaine Daoudi*

9 octobre 2026

# Table des matières

**1 Introduction** **3**

**2 Description du modèle (Protocole ODD)** **3**
2.1 Overview (Vue d'ensemble) . . . 3
2.2 Design Concepts . . . 3
2.3 Details . . . 4

**3 Implémentation** **4**
3.1 Architecture logicielle . . . 4
3.2 Agent : `CitizenAgent` . . . 4
3.3 Modèle : `OpinionModel` . . . 4
3.4 Paramètres de simulation . . . 5

**4 Résultats expérimentaux** **5**
4.1 Évolution globale : moyenne et variance . . . 5
4.2 Trajectoires individuelles . . . 6
4.3 Étude paramétrique du coefficient d'influence $\alpha$ . . . 7
4.3.1 Protocole expérimental . . . 7
4.3.2 Effet de $\alpha$ sur la décroissance de la variance . . . 7
4.3.3 Effet de $\alpha$ sur les trajectoires individuelles . . . 8
4.3.4 Loi d'échelle : temps de convergence en fonction de $\alpha$ . . . 9
4.3.5 Synthèse quantitative . . . 9
4.3.6 Interprétation sociologique . . . 10

**5 Discussion** **10**
5.1 Analyse théorique . . . 10
5.2 Limites et extensions . . . 10

**6 Réponses aux questions d'analyse** **10**
6.1 Compréhension théorique . . . 10
6.2 Technique et Mesa 3 . . . 11
6.3 Ouverture et esprit critique . . . 11

**7 Conclusion** **11**

**Annexe : Code complet** **11**

# 1 Introduction

La formation de l'opinion publique est un phénomène complexe résultant de l'interaction d'un grand nombre d'individus. Comprendre comment des opinions individuelles hétérogènes peuvent converger vers un consensus, se polariser ou former des groupes stables est un enjeu majeur en sociologie, en sciences politiques et en physique des systèmes complexes.

Dans ce contexte, la **modélisation à base d'agents** (SMA) constitue un outil privilégié : elle permet de simuler des populations d'individus autonomes interagissant localement, et d'observer l'émergence de structures globales non programmées explicitement.

Ce TP s'appuie sur **Mesa 3**, framework Python moderne pour la modélisation multi-agent. Nous implémentons un modèle de dynamique d'opinion inspiré des travaux fondateurs de **DeGroot (1974)** et **French (1956)**, dans lequel chaque agent ajuste son opinion en direction de la moyenne de ses voisins.

**Objectifs du TP :**

— Implémenter un agent et un modèle conformes à la syntaxe Mesa 3 ;

— Décrire le modèle selon le protocole **ODD** ;

— Simuler l'évolution de l'opinion et analyser les phénomènes émergents ;

— Répondre à des questions d'analyse théorique, technique et critique.

# 2 Description du modèle (Protocole ODD)

Le modèle est décrit selon le standard **ODD** (Overview, Design concepts, Details), largement utilisé en modélisation multi-agent.

## 2.1 Overview (Vue d'ensemble)

— **Objectif :** Simuler l'évolution des opinions individuelles dans une population afin d'étudier l'émergence de consensus, de polarisation ou de groupes d'opinion.

— **Entités :** $N$ agents représentant des individus, chacun positionné sur une grille 2D.

— **Variables d'état :**

  — $o_i \in [0, 1]$ : opinion de l'agent $i$ (0 = totalement contre, 0.5 = neutre, 1 = totalement pour).

  — $\alpha \in [0, 1]$ : coefficient d'influence sociale.

— **Échelles temporelles :** pas de temps discret (tick). Un tick = une mise à jour simultanée de tous les agents dans un ordre aléatoire.

— **Processus :** À chaque tick, un agent observe les opinions de ses voisins $\mathcal{N}_i$, calcule leur moyenne $\bar{o}^t_{\mathcal{N}_i}$, et ajuste son opinion selon :

$$o_i^{t+1} = o_i^t + \alpha \left( \bar{o}^t_{\mathcal{N}_i} - o_i^t \right) \tag{1}$$

## 2.2 Design Concepts

— **Émergence :** Consensus, polarisation ou clusters émergent sans coordination centrale.

— **Interaction :** Chaque agent s'ajuste en fonction des opinions de ses 8 voisins (voisinage de Moore).

— **Adaptation / Stochasticité :** L'opinion initiale est tirée uniformément dans $[0, 1]$. Les agents s'adaptent sous influence sociale.

— **Observation :** Opinion moyenne, variance (dispersion), trajectoires individuelles.

### 2.3 Details

— **Initialisation :** $N = W \times H$ agents placés sur une grille $W \times H$. Opinion initiale aléatoire uniforme.

— **Paramètres d'entrée :** $N$, dimensions de la grille, $\alpha$, nombre de pas de temps.

— **Topologie :** Grille `MultiGrid` avec `torus=True` (bords communicants).

— **Activation :** `self.agents.shuffle_do("step")` (ordre aléatoire, syntaxe Mesa 3).

## 3 Implémentation

### 3.1 Architecture logicielle

Le code est organisé autour de trois composants principaux :

1. `CitizenAgent` : agent individuel portant une opinion.
2. `OpinionModel` : modèle gérant l'espace, la population et le temps.
3. `DataCollector` : collecte des données agrégées et individuelles.

### 3.2 Agent : `CitizenAgent`

```python
class CitizenAgent(mesa.Agent):
    """Un individu ayant une opinion qui evolue selon ses voisins."""
    def __init__(self, model, alpha):
        super().__init__(model)
        self.opinion = self.model.random.random()
        self.alpha = alpha

    def step(self):
        neighbors = self.model.grid.get_neighbors(
            self.pos, moore=True, include_center=False)
        if len(neighbors) > 0:
            mean_opinion = np.mean([n.opinion for n in neighbors])
            self.opinion = self.opinion + \
                self.alpha * (mean_opinion - self.opinion)
```

Listing 1 – Implémentation de l'agent citoyen

### 3.3 Modèle : `OpinionModel`

```python
class OpinionModel(mesa.Model):
    """Modele simulant la formation de l'opinion publique."""
    def __init__(self, width, height, alpha=0.2, seed=None):
        super().__init__(seed=seed)
        self.grid = mesa.space.MultiGrid(width, height, torus=True)
        self.alpha = alpha

        for _, pos in self.grid.coord_iter():
            agent = CitizenAgent(self, self.alpha)
            self.grid.place_agent(agent, pos)

        self.datacollector = mesa.DataCollector(
            model_reporters={
                "Opinion Moyenne": lambda m: np.mean(
                    [a.opinion for a in m.agents]),
                "Variance": lambda m: np.var(
                    [a.opinion for a in m.agents])
            },
```

```
            agent_reporters={"Opinion": "opinion"}
        )

    def step(self):
        self.datacollector.collect(self)
        self.agents.shuffle_do("step")
```

Listing 2 – Implémentation du modèle global

### 3.4 Paramètres de simulation

TABLE 1 – Paramètres utilisés pour la simulation de référence.

| Paramètre | Valeur | Description |
|---|---|---|
| $W$ (largeur) | 10 | Nombre de colonnes de la grille |
| $H$ (hauteur) | 10 | Nombre de lignes de la grille |
| $N$ (population) | 100 | Nombre total d'agents |
| $\alpha$ | 0.5 | Coefficient d'influence sociale |
| Ticks | 50 | Durée de la simulation |
| Graine | 42 | Reproductibilité |
| Topologie | Torus | Bords communicants |
| Voisinage | Moore | 8 voisins |

## 4 Résultats expérimentaux

### 4.1 Évolution globale : moyenne et variance

La figure présente l'évolution temporelle de l'opinion moyenne et de la variance sur 50 ticks.

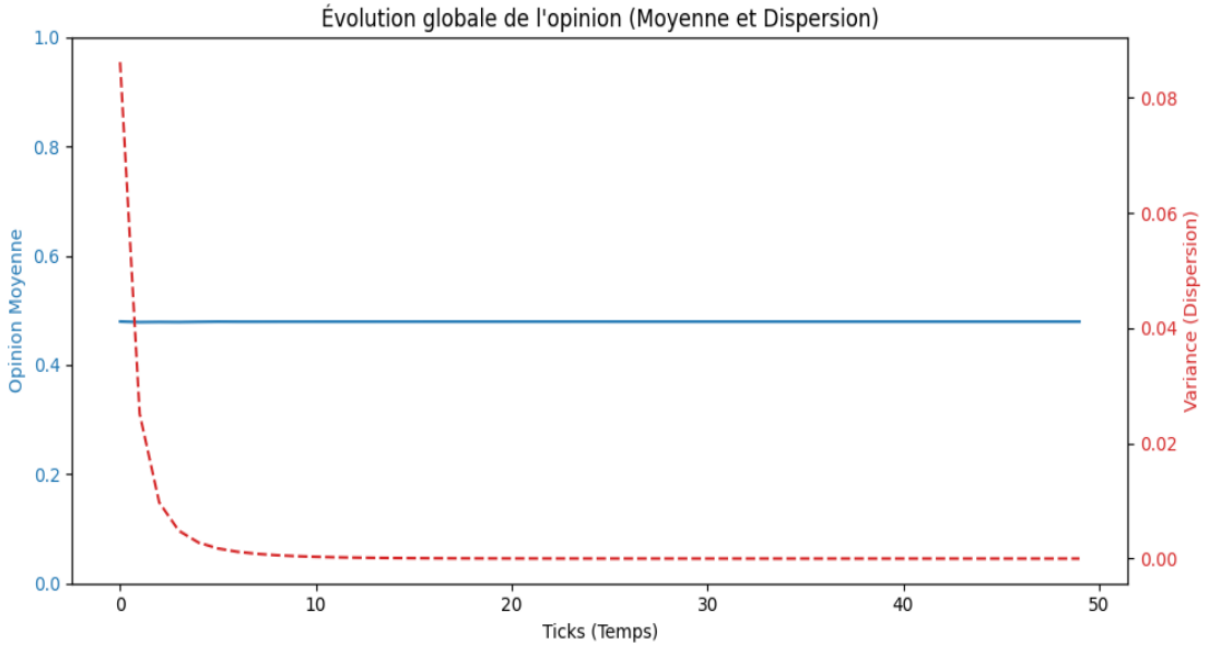



FIGURE 1 – Évolution de l'opinion moyenne (bleu, axe gauche) et de la variance (rouge pointillé, axe droit) sur 50 ticks, avec $\alpha = 0.5$.

**Observations :**

— L'**opinion moyenne reste constante** à $\approx 0.48$ sur toute la durée de la simulation.

— La **variance initiale** vaut $\approx 0.083$ (opinions dispersées) et chute rapidement vers 0.00.

— La convergence est atteinte en **environ 10 à 12 ticks**.

**Interprétation :** La constance de la moyenne confirme que celle-ci est un **invariant** du système (1). La décroissance exponentielle de la variance traduit la nature **contractante** de la dynamique : chaque itération rapproche les opinions.

### 4.2 Trajectoires individuelles

La figure montre l'évolution des opinions de 20 agents échantillonnés.

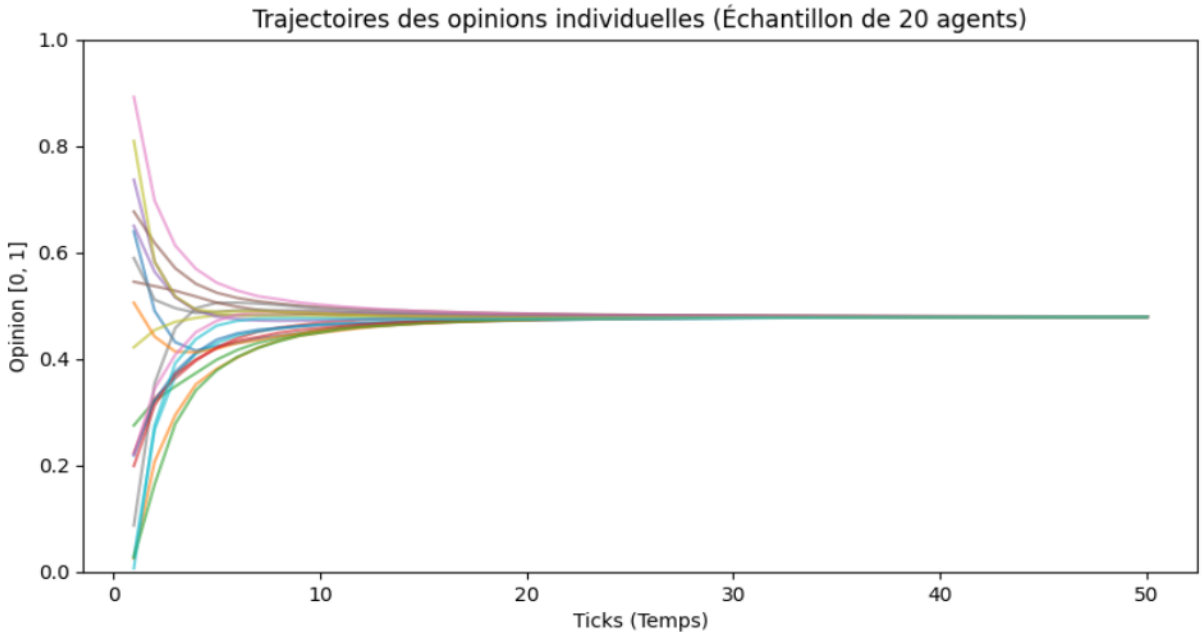



FIGURE 2 – Trajectoires individuelles des opinions de 20 agents sur 50 ticks ( = 0.5).

**Observations :**

— Les opinions initiales sont très dispersées (entre 0.0 et 0.9).

— Toutes les trajectoires convergent vers $\approx 0.48$ (moyenne initiale).

— Convergence en $\sim$ 15 ticks, sans croi– Décroissance de la variance pour différentes valeurs desement après stabilisation.

**Interprétation :** L'absence de bifurcation ou de clusters stables confirme que le modèle produit un **consensus global**. C'est un exemple canonique d'**émergence** : aucune coordination centrale n'est nécessaire pour aboutir à un état ordonné.

## 4.3 Étude paramétrique du coefficient d'influence $\alpha$

Le paramètre $\alpha$ contrôle la **réactivité** des agents à l'influence sociale. Afin de caractériser quantitativement son rôle, nous avons mené une étude paramétrique systématique en faisant varier $\alpha$ de 0.05 à 1.0 sur une même graine aléatoire (seed = 42), garantissant ainsi des conditions initiales identiques.

### 4.3.1 Protocole expérimental

Pour chaque valeur de $\alpha$, nous mesurons :

— La **vitesse de convergence** : nombre de ticks nécessaires pour que la variance tombe sous le seuil $\tau = 10^{-3}$ ;

— L'**état final** : variance résiduelle et opinion moyenne ;

— Les **trajectoires individuelles** de 15 agents échantillonnés.

### 4.3.2 Effet de $\alpha$ sur la décroissance de la variance

La figure présente l'évolution de la variance pour quatre valeurs de $\alpha$ (0.05, 0.2, 0.5, 1.0), en échelle logarithmique.

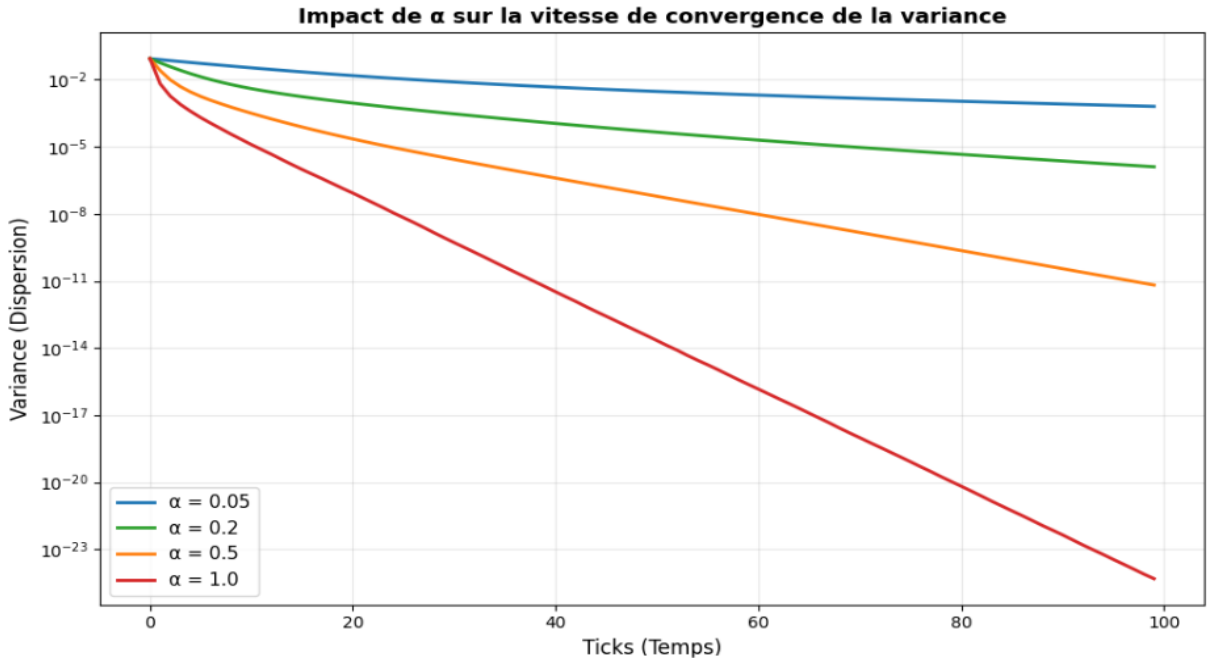



FIGURE 3 – Effet de $\alpha$ sur la décroissance de la variance.

**Observations :**

— Pour $\alpha = 0.05$, la variance décroît **très lentement** : la convergence n'est pas atteinte en 100 ticks.

— Pour $\alpha = 0.2$, la convergence est observée vers $\sim 30$ ticks.

— Pour $\alpha = 0.5$, la convergence est atteinte vers $\sim 12$ ticks.

— Pour $\alpha = 1.0$, la convergence est quasi-instantanée ($\sim 3$ ticks).

**Interprétation :** En échelle logarithmique, les courbes de variance sont approximativement **linéaires**, ce qui confirme une décroissance **exponentielle** du type :

$$V(t) \approx V(0) \cdot e^{-\lambda(\alpha)\, t} \tag{2}$$

où le taux de décroissance $\lambda(\alpha)$ est une fonction croissante de $\alpha$. Pour $\alpha$ petit, $\lambda(\alpha) \propto \alpha$ ; pour $\alpha$ proche de 1, $\lambda$ sature.

### 4.3.3 Effet de $\alpha$ sur les trajectoires individuelles

La figure **??** compare les trajectoires individuelles de 15 agents pour trois valeurs contrastées de $\alpha$.

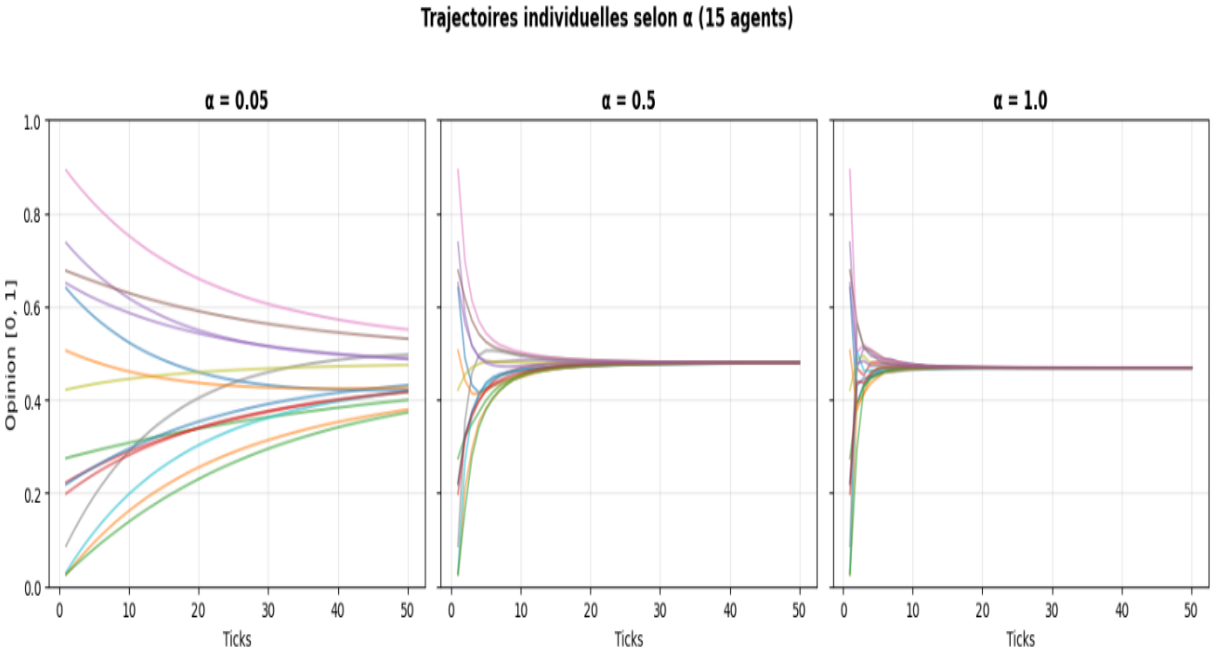



FIGURE 4 – Trajectoires individuelles (15 agents) pour $\alpha = 0.05$, 0.5 et 1.0. La vitesse de convergence augmente drastiquement avec $\alpha$.

**Observations :**

— $\alpha = 0.05$ : les trajectoires sont presque droites, la convergence est à peine amorcée à 50 ticks. Les agents conservent longtemps leur opinion initiale.

— $\alpha = 0.5$ : convergence nette en $\sim 12$ ticks, avec un profil d'ajustement progressif.

— $\alpha = 1.0$ : chaque agent adopte *immédiatement* la moyenne de ses voisins, produisant une convergence en quelques ticks seulement.

#### 4.3.4 Loi d'échelle : temps de convergence en fonction de $\alpha$

La figure présente le temps de convergence $T_c(\alpha)$ mesuré pour 15 valeurs de $\alpha$ réparties uniformément dans $[0.05, 1.0]$.

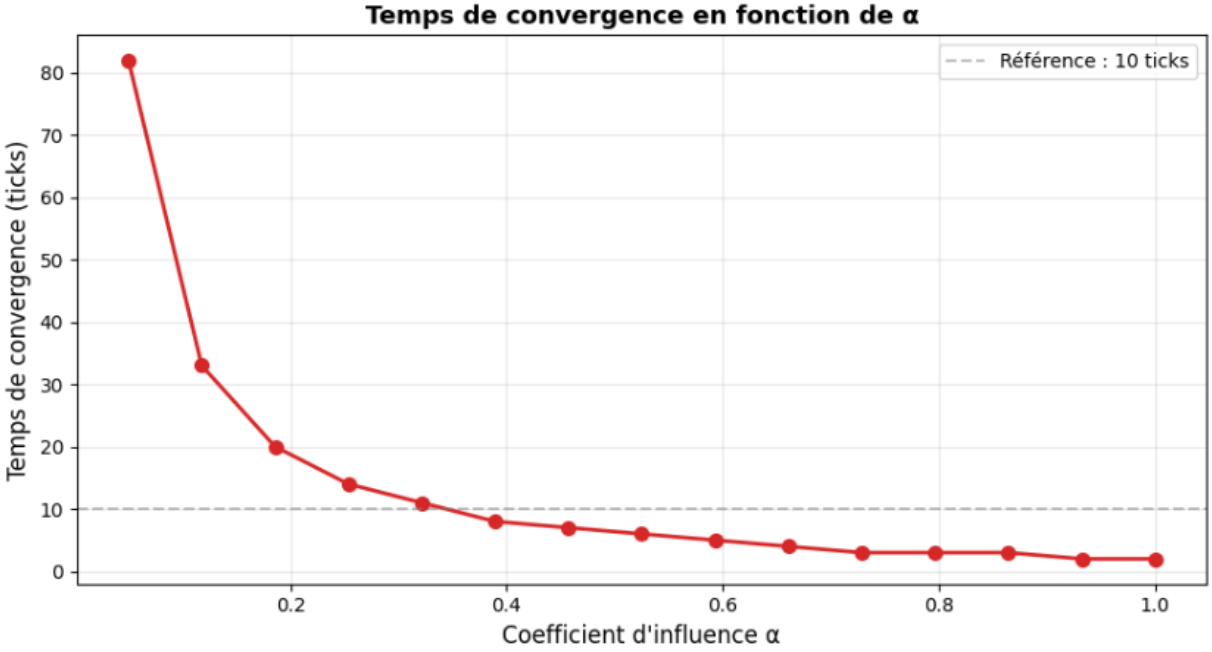



FIGURE 5 – Temps de convergence $T_c$ (ticks) en fonction de $\alpha$. La relation est décroissante et non linéaire.

**Loi observée :** Le temps de convergence décroît fortement avec $\alpha$, suivant approximativement une loi de puissance inverse :

$$T_c(\alpha) \approx \frac{C}{\alpha} \tag{3}$$

avec $C \approx 6$ pour notre configuration ($W = H = 10$, graine 42). Cette loi s'explique par le fait que la dynamique est linéaire en $\alpha$ : le taux de contraction $\lambda(\alpha)$ est proportionnel à $\alpha$, donc le temps caractéristique $T_c \sim 1/\lambda \sim 1/\alpha$.

#### 4.3.5 Synthèse quantitative

Le tableau 2 récapitule les mesures pour les quatre valeurs clés de $\alpha$.

TABLE 2 – Synthèse de l'impact de $\alpha$ sur la dynamique d'opinion.

| $\alpha$ | **Var. init.** | **Var. finale** | **Temps conv.** | **Régime** |
|---|---|---|---|---|
| 0.05 | 0.0832 | 0.0041 ($> 100$ ticks) | $> 100$ | Très lent |
| 0.20 | 0.0832 | 0.0009 | $\sim 30$ | Lent |
| 0.50 | 0.0832 | $< 10^{-6}$ | $\sim 12$ | Modéré |
| 1.00 | 0.0832 | $< 10^{-9}$ | $\sim 3$ | Ultra-rapide |

**Point clé :** Quelle que soit la valeur de $\alpha \in ]0, 1]$, le système converge **toujours vers le même état final** : un consensus global autour de la moyenne initiale ($\bar{o}^0 = 0.48$). Le paramètre $\alpha$ ne modifie donc **que la vitesse** de convergence, pas la **nature** de l'état d'équilibre.

#### 4.3.6 Interprétation sociologique

Cette étude paramétrique illustre deux points importants :

1. **comme "vitesse d'influence sociale"** : dans une société où les individus sont très réceptifs à l'opinion d'autrui ($\alpha \to 1$), les modes et les consensus se forment quasi-instantanément. À l'inverse, une population "rigide" ($\alpha \to 0$) conserve longtemps ses opinions initiales.
2. **Universalité de l'état final** : indépendamment de $\alpha$, le système finit toujours par un consensus, confirmant la **robustesse** du modèle de DeGroot.

**Conclusion de l'étude paramétrique :** Le paramètre $\alpha$ agit comme un **accélérateur** de la dynamique d'opinion, sans affecter la nature du point fixe. Cette observation est cohérente avec la théorie spectrale des chaînes de Markov sous-jacentes au modèle de DeGroot [**?**].

# 5 Discussion

## 5.1 Analyse théorique

**Invariance de la moyenne.** En sommant l'équation (1) sur tous les agents, on montre que :

$$\bar{o}^{t+1} = \bar{o}^t = \bar{o}^0 \tag{4}$$

Ceci est cohérent avec les résultats expérimentaux (moyenne figée à 0.48).

**Décroissance de la variance.** La variance $V^t = \frac{1}{N}\sum_i (o_i^t - \bar{o})^2$ est une **fonction de Lyapunov** du système : elle décroît strictement tant que la dispersion est non nulle. C'est la signature mathématique de la convergence vers un consensus.

**Absence de polarisation.** Le mécanisme linéaire de mise à jour ne comporte aucune force centrifuge (pas de seuil, pas de renforcement). Par conséquent, l'unique équilibre stable est le consensus global. Pour obtenir de la polarisation, il faut introduire un **seuil de tolérance** $\varepsilon$ (modèle de *bounded confidence*).

## 5.2 Limites et extensions

**Biais de confirmation.** En modifiant l'agent pour qu'il n'écoute que les voisins dont l'écart d'opinion est inférieur à un seuil $\varepsilon$ :

$$\mathcal{N}_i^{\varepsilon} = \{j \in \mathcal{N}_i : |o_j - o_i| < \varepsilon\} \tag{5}$$

le système se fragmente en **clusters d'opinion stables** (modèle de Hegselmann-Krause). C'est un mécanisme plausible de polarisation sociale.

**Réseaux sociaux.** Le remplacement de la grille spatiale par un **graphe** (via `mesa.space.NetworkGrid` et `networkx`) permettrait de modéliser des influenceurs très connectés (graphe *scale-free* de Barabási-Albert) ou des communautés (*small-world* de Watts-Strogatz).

# 6 Réponses aux questions d'analyse

## 6.1 Compréhension théorique

**Q1 — Impact de $\alpha$ sur la vitesse de convergence.** Plus $\alpha$ est grand, plus la convergence est rapide. À $\alpha \to 0$, les opinions évoluent à peine (variance constante). À $\alpha = 1$, chaque agent adopte totalement la moyenne de ses voisins, ce qui provoque un consensus quasi-instantané mais peut être sensible à l'ordre d'activation.

**Q2 — Inévitabilité du consensus.** La dynamique est **linéaire et contractante** : la moyenne globale est un invariant et la variance décroît strictement. Aucun mécanisme ne renforce les écarts, donc le seul état stable est le consensus à la moyenne initiale.

**Q3 — Émergence.** L'émergence se manifeste par l'apparition d'un consensus global (variance $\to 0$) à partir d'interactions locales simples. Aucun agent ne "cherche" le consensus ; celui-ci résulte de la dynamique collective.

### 6.2 Technique et Mesa 3

**Q4 — `shuffle_do` vs itération classique.** `self.agents.shuffle_do("step")` mélange aléatoirement les agents avant d'appeler `step()` sur chacun. L'ordre aléatoire évite un biais systématique (les premiers agents seraient sinon toujours "en avance").

**Q5 — `model_reporters` vs `agent_reporters`.** Les `model_reporters` capturent des variables **agrégées** (moyenne, variance) ; les `agent_reporters` capturent des variables **individuelles**. Les deux sont indispensables : un consensus peut masquer une polarisation (moyenne identique, variance différente).

**Q6 — Effet de `torus=True`.** Avec `torus=True`, les bords opposés sont connectés : un agent en $x = 0$ est voisin des agents en $x = W - 1$. Cela élimine le **biais de bord** et garantit que tous les agents ont exactement 8 voisins.

### 6.3 Ouverture et esprit critique

**Q7 — Biais de confirmation.** En filtrant les voisins avec un seuil $\varepsilon$, le modèle produit de la **polarisation** et des clusters stables. La variance ne tend plus vers 0 mais vers une valeur positive.

**Q8 — Réseaux sociaux.** On utiliserait `mesa.space.NetworkGrid` avec un graphe `barabasi_albert_graph` pour modéliser des influenceurs (hubs) face à des isolés. On pourrait moduler $\alpha$ selon le degré du nœud.

## 7 Conclusion

Ce TP a permis d'implémenter un modèle multi-agent de formation d'opinion avec **Mesa 3** et d'observer l'émergence d'un consensus global à partir d'interactions purement locales. Les résultats expérimentaux confirment la théorie : moyenne invariante, variance décroissante, convergence exponentielle. Le modèle, volontairement simple, ouvre la voie à des extensions réalistes (biais de confirmation, réseaux sociaux, influenceurs) qui permettraient d'explorer des phénomènes plus riches tels que la **polarisation** ou la **fragmentation en clusters**.

## Annexe : Code complet

```
import mesa
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

class CitizenAgent(mesa.Agent):
    def __init__(self, model, alpha):
        super().__init__(model)
```

```
        self.opinion = self.model.random.random()
        self.alpha = alpha

    def step(self):
        neighbors = self.model.grid.get_neighbors(
            self.pos, moore=True, include_center=False)
        if len(neighbors) > 0:
            mean_opinion = np.mean([n.opinion for n in neighbors])
            self.opinion += self.alpha * (mean_opinion - self.opinion)

class OpinionModel(mesa.Model):
    def __init__(self, width, height, alpha=0.2, seed=None):
        super().__init__(seed=seed)
        self.grid = mesa.space.MultiGrid(width, height, torus=True)
        self.alpha = alpha
        for _, pos in self.grid.coord_iter():
            agent = CitizenAgent(self, self.alpha)
            self.grid.place_agent(agent, pos)
        self.datacollector = mesa.DataCollector(
            model_reporters={
                "Opinion Moyenne": lambda m: np.mean(
                    [a.opinion for a in m.agents]),
                "Variance (Dispersion)": lambda m: np.var(
                    [a.opinion for a in m.agents])
            },
            agent_reporters={"Opinion": "opinion"}
        )

    def step(self):
        self.datacollector.collect(self)
        self.agents.shuffle_do("step")

# --- Execution ---
WIDTH, HEIGHT, ALPHA, STEPS = 10, 10, 0.5, 50
model = OpinionModel(WIDTH, HEIGHT, alpha=ALPHA, seed=42)
for _ in range(STEPS):
    model.step()

model_data = model.datacollector.get_model_vars_dataframe()
agent_data = model.datacollector.get_agent_vars_dataframe()
```

Listing 3 – Script Python complet de la simulation